



UNIDAD 3

Desarrollo Frontend con React

Tecnicatura Universitaria en Programación 2026



CONTENIDO

01

Introducción a
React

02

Componentes y
JSX

03

Props y
estados

04

Consumo de
APIs

¿Qué es React JS?

Es una librería de Javascript para construir interfaces de usuario. Las interfaces son la forma que tiene el usuario para interactuar con el sistema.

React Js nos permite crear aplicaciones web, aplicaciones móviles y aplicaciones de escritorio.



Basado en componentes

- Las interfaces de usuario que crearemos están basadas en componentes.
- Un componente es un conjunto de elementos que cumplen una función específica. Por ejemplo, un botón.
- Esto quiere decir que puedo tener componentes compuestos por otros componentes.
- Las aplicaciones de React están hechas a partir de componentes. Un componente es una pieza de UI (siglas en inglés de interfaz de usuario) que tiene su propia lógica y apariencia. Un componente puede ser tan pequeño como un botón, o tan grande como toda una página.

Crear una aplicación React desde cero

El primer paso es instalar una herramienta de compilación

Vite

Vite es una herramienta de compilación cuyo objetivo es proporcionar una experiencia de desarrollo más rápida y sencilla para los proyectos web modernos.

Ejecutar en la terminal: `npm create vite@latest my-app -- --template react`

Crear y anidar componentes

Los componentes de React son funciones de JavaScript

```
function MyButton() {  
  return (  
    <button>Soy un botón</button>  
  );  
}
```

Crear y anidar componentes

Este se puede anidar en otro componente

```
export default function MyApp() {  
  return (  
    <div>  
      <h1>Bienvenido a mi aplicación</h1>  
      <MyButton />  
    </div>  
  );  
}
```

Las palabras clave `export default` especifican el componente principal en el archivo.

Nota que `<MyButton />` empieza con mayúscula.

Los nombres de los componentes de React siempre deben comenzar con mayúscula, mientras las etiquetas HTML deben estar minúsculas.

Escribir marcado con JSX (JS XML)

JSX es una **extensión de sintaxis de JavaScript** que permite escribir estructuras con apariencia de HTML dentro del código JavaScript.

JSX es más estricto que HTML. Tienes que cerrar etiquetas como `<br />`. **Tu componente tampoco puede devolver múltiples etiquetas de JSX.** Debes envolverlas en un padre compartido, como `<div>...</div>` o en un envoltorio vacío `<>...</>`:

```
function AboutPage() {  
  return (  
    <>  
      <h1>Acerca de</h1>  
      <p>Hola.<br />¿Cómo vas?</p>  
    </>  
  );  
}
```


Añadir estilos

En React, especificas una clase de CSS con **className**. Funciona de la misma forma que el atributo class de HTML:

```
<img className="avatar" />
```

Luego escribes las reglas CSS para esa clase en un archivo CSS aparte

Renderizado condicional

En React, no hay una sintaxis especial para escribir condicionales. En cambio, usarás las mismas técnicas que usas al escribir código regular de JavaScript. Por ejemplo, puedes usar una sentencia `if` para incluir JSX condicionalmente:

```
let content;
if (isLoggedIn) {
  content = <AdminPanel />;
} else {
  content = <LoginForm />;
}
return (
  <div>
    {content}
  </div>
);
```

Renderizado condicional

Si prefieres un código más compacto, puedes utilizar el operador ? condicional. A diferencia de if, funciona dentro de JSX:

```
<div>
  {isLoggedIn ? (
    <AdminPanel />
  ) : (
    <LoginForm />
  )}
</div>
```

Renderizado condicional

Cuando no necesites la rama else, puedes también usar la sintaxis lógica &&, más breve:

```
<div>  
  {isLoggedIn && <AdminPanel />}  
</div>
```

Renderizado de listas

Dependerás de funcionalidades de JavaScript como los bucles for y la función `map()` de los arreglos para renderizar listas de componentes.

Ejemplo de array

```
const products = [  
  { title: 'Col', id: 1 },  
  { title: 'Ajo', id: 2 },  
  { title: 'Manzana', id: 3 },  
];
```

Renderizado de listas

Dentro de tu componente, utiliza la función `map()` para transformar el arreglo de productos en un arreglo de elementos `<li>`:

```
const listItems = products.map(product =>
  <li key={product.id}>
    {product.title}
  </li>
);

return (
  <ul>{listItems}</ul>
);
```

Nota que `<li>` tiene un atributo `key` (llave). Para cada elemento en una lista, debes pasar una cadena o un número que identifique ese elemento de forma única entre sus hermanos.

Responder a eventos

Puedes responder a eventos declarando funciones controladoras de eventos dentro de tus componentes:

```
function MyButton() {  
  function handleClick() {  
    alert('¡Me hiciste clic!');  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Hazme clic  
    </button>  
  );  
}
```

Actualizar la pantalla

A menudo, queremos que el componente “recuerde” alguna información y la muestre. Por ejemplo, contar el número de veces que hiciste click en un botón. Para lograrlo, añade estado a tu componente.

Primero, importa useState de React:

```
import { useState } from 'react';
```

Ahora puedes declarar una variable de estado dentro de tu componente:

```
function MyButton() {  
  const [count, setCount] = useState(0);  
  // ...  
}
```


Actualizar la pantalla

Obtendrás dos cosas de `useState`: el estado actual (`count`), y la función que te permite actualizarlo (`setCount`). Puedes nombrarlos de cualquier forma, pero la convención es llamarlos como `[algo, setAlgo]`.

La primera vez que se muestra el botón, `count` será 0 porque pasaste 0 a `useState()`. Cuando quieras cambiar el estado llama a `setCount()` y pásale el nuevo valor.

Actualizar la pantalla

```
function MyButton() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    setCount(count + 1);  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Hiciste clic {count} veces  
    </button>  
  );  
}
```

Uso de Hooks

Las funciones que comienzan con use se llaman Hooks. useState es un Hook nativo dentro de React. **Un Hook es una función de React que permite agregar memoria y comportamiento a un componente funcional.**

Los Hooks son más restrictivos que las funciones regulares. Solo puedes llamar a los Hooks en el primer nivel de tus componentes (u otros Hooks). Si quisieras utilizar useState en una condicional o en un bucle, extrae un nuevo componente y ponlo ahí.

Anatomía del useState

Cuando llamamos al useState, le estamos diciendo a React que debe recordar algo:

```
const [index, setIndex] = useState(0);
```

En este caso, queremos que React recuerde el index.

El único argumento para useState es el valor inicial de su variable de estado. En este ejemplo, el valor inicial de index se establece en 0 con useState(0).

Ejemplo

Anatomía del useState

Cada vez que el componente se renderiza, el useState devuelve un array que contiene dos valores:

- 1- La variable de estado (index) con el valor que almacenaste.
- 2- La función que establece el estado (setIndex) que puede actualizar la variable de estado y alertar a React para que renderice el componente nuevamente.

```
const [index, setIndex] = useState(0);
```

Anatomía del useState

1. **Tu componente se renderiza por primera vez.** Debido a que pasamos 0 a useState como valor inicial para index, esto devolverá [0, setIndex]. React recuerda que 0 es el último valor de estado.
2. **Actualizas el estado.** Cuando un usuario hace clic en el botón, llama a setIndex(index + 1). index es 0, por lo tanto es setIndex(1). Esto le dice a React que recuerde que index es 1 ahora y ejecuta otro renderizado.
3. **El componente se renderiza por segunda vez.** React todavía ve useState(0), pero debido a que React *recuerda* que estableció index en 1, devuelve [1, setIndex] en su lugar.
4. ¡Y así sucesivamente!

Colocar múltiples variables de estado a un componente

Podemos tener más de una variable de estado de diferentes tipos en un componente. Este componente tiene dos variables de estado, un `index` numérico y un `showMore` booleano que se activa al hacer clic en “Mostrar detalles”:

Ejemplo

Es una buena idea tener múltiples variables de estado si no se encuentran relacionadas entre sí, como `index` y `showMore` en este ejemplo. Pero si encontramos que a menudo cambiamos dos variables de estado juntas, podría ser mejor combinarlas en una sola. Por ejemplo, si tenemos un formulario con muchos campos, es más conveniente tener una única variable de estado que contenga un objeto que una variable de estado por campo. [Leer más](#)

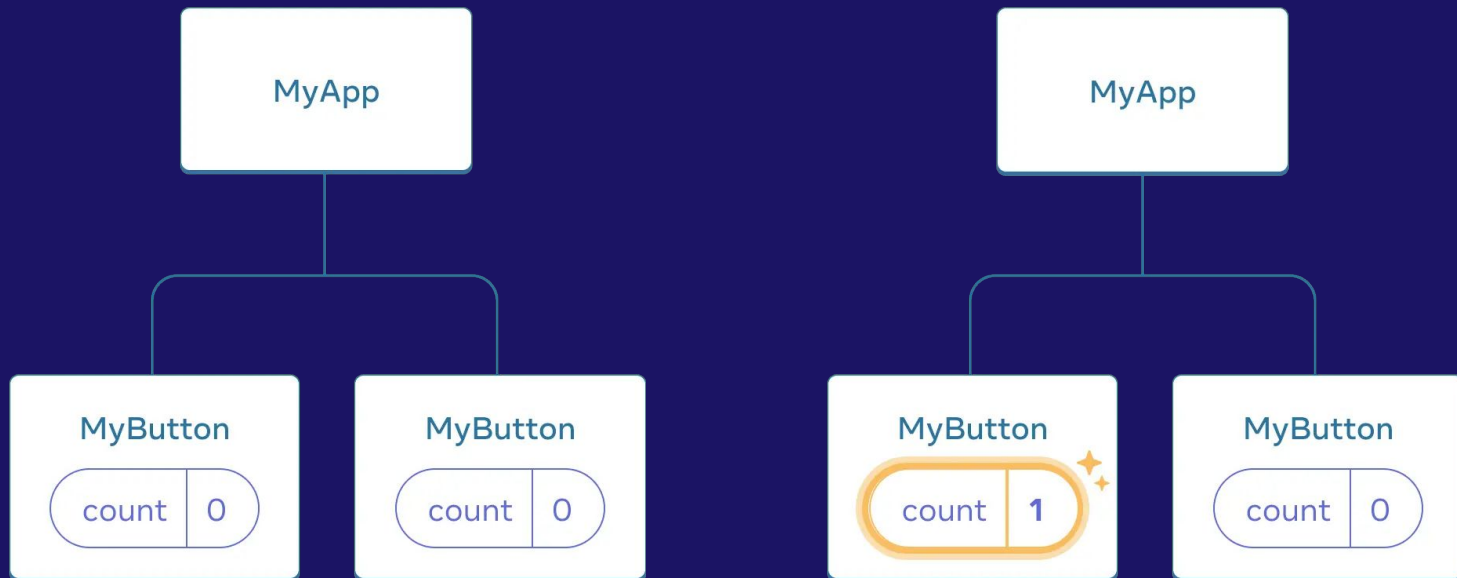
El estado es aislado y privado

El estado es local para una instancia de un componente en la pantalla. En otras palabras, si renderizas el mismo componente dos veces, ¡cada copia tendrá un estado completamente aislado! Cambiar uno de ellos no afectará al otro.

¿Qué pasaría si quisieramos que ambas galerías mantuvieran sus estados sincronizados? La forma correcta de hacerlo en React es eliminar el estado de los componentes secundarios y agregarlo a su padre más cercano.

Compartir datos entre componentes

En el ejemplo anterior, cada MyButton tenía su propio count independiente, y cuando se hace click en cada botón, solo el count de ese botón cambia:



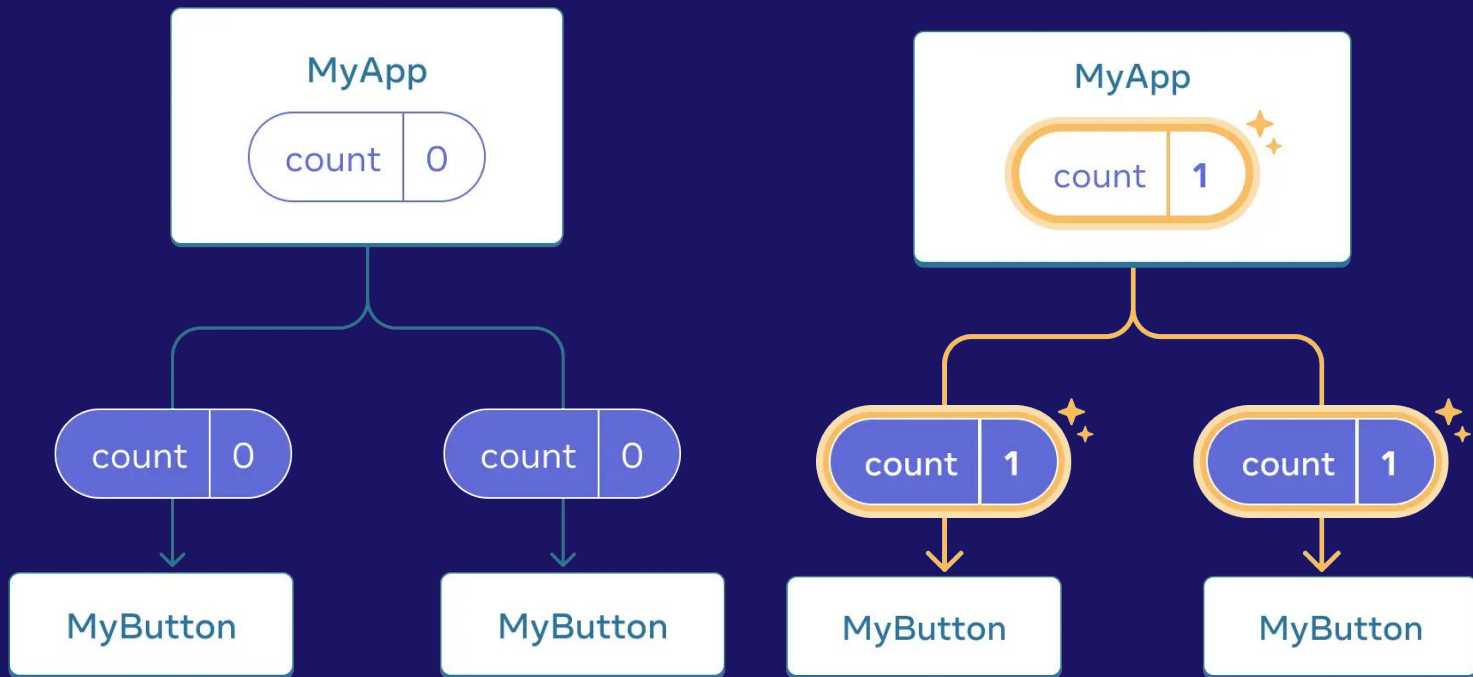
Compartir datos entre componentes

Sin embargo, a menudo necesitas que los componentes compartan datos y se actualicen siempre en conjunto.

Para hacer que ambos componentes MyButton muestren el mismo count y se actualicen juntos, necesitas mover el estado de los botones individuales “hacia arriba” al componente más cercano que los contiene a todos.

En este ejemplo, es MyApp:

Compartir datos entre componentes



Ahora cuando haces click en cualquiera de los botones, count en MyApp cambiará, lo que causará que cambien ambos counts en MyButton

```
export default function MyApp() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    setCount(count + 1);  
  }  
  
  return (  
    <div>  
      <h1>Contadores que se actualizan separadamente</h1>  
      <MyButton />  
      <MyButton />  
    </div>  
  );  
}  
  
function MyButton() {  
  // ... estamos moviendo el código de aquí ...  
}
```

Luego, *pasa el estado hacia abajo* desde MyApp hacia cada MyButton, junto con la función compartida para controlar el evento de clic. Puedes pasar la información a MyButton usando las llaves de JSX,

```
export default function MyApp() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    setCount(count + 1);  
  }  
  
  return (  
    <div>  
      <h1>Contadores que se actualizan juntos</h1>  
      <MyButton count={count} onClick={handleClick} />  
      <MyButton count={count} onClick={handleClick} />  
    </div>  
  );  
}
```

La información que pasas hacia abajo se llaman props. Ahora el componente MyApp contiene el estado count y el controlador de evento handleClick, y pasa ambos hacia abajo como props a cada uno de los botones.

Finalmente, cambia MyButton para que lea las props que le pasaste desde el componente padre:

```
function MyButton({ count, onClick }) {  
  return (  
    <button onClick={onClick}>  
      Hiciste clic {count} veces  
    </button>  
  );  
}
```

Cuando haces clic en el botón, el controlador `onClick` se dispara. A la prop `onClick` de cada botón se le asignó la función `handleClick` dentro de `MyApp`, de forma que el código dentro de ella se ejecuta. Ese código llama a `setCount(count + 1)`, que incrementa la variable de estado `count`. El nuevo valor de `count` se pasa como prop a cada botón, y así todos muestran el nuevo valor.

Esto se llama “levantar el estado”. Al mover el estado hacia arriba, lo compartimos entre componentes.

Más sobre props

Las props son los datos que se pasan a un elemento JSX. Por ejemplo, `className`, `src`, `alt`, `width` y `height` son algunas de las props que se pueden pasar a un elemento `<img>`:

```
function Avatar() {  
  return (  
      
  );  
}  
  
export default function Profile() {  
  return (  
    <Avatar />  
  );  
}
```


Pasar props a un componente

En este código, el componente Profile no está pasando ninguna prop a su componente hijo, Avatar:

```
export default function Profile() {  
  return (  
    <Avatar />  
  );  
}
```

Paso 1: Pasar props al component hijo

Primero, pasa algunas props al elemento Avatar. Por ejemplo, vamos a asignar dos props: person (un objeto) y size (un número):

```
export default function Profile() {  
  return (  
    <Avatar  
      person={{ name: 'Lin Lanying', imageId: '1bX5QH6' }}  
      size={100}  
    />  
  );  
}
```

Paso 2: Acceder a props dentro del componente hijo

Puedes acceder a estas props especificando sus nombres `person`, `size` separados por comas dentro de (`{` y `}`) justo después de `function Avatar`. Esto te permitirá utilizarlas dentro del código de `Avatar` como si fueran variables.

```
function Avatar({ person, size }) {  
  // puedes acceder a los valores de person y size desde aquí  
}
```

Ahora puedes configurar Avatar para que se renderice de diferentes maneras con distintas props.

Ejemplo

Asignar un valor predeterminado para una prop

Si quieres asignar un valor predeterminado para una prop en caso de que no se especifique ningún valor, puedes hacerlo mediante la desestructuración colocando = seguido del valor predeterminado justo después del parámetro:

```
function Avatar({ person, size = 100 }) {  
  // ...  
}
```

Ahora, si renderizas `<Avatar person={...} />` sin la prop size, el valor de size se establecerá automáticamente en 100.

Pasar JSX como hijos

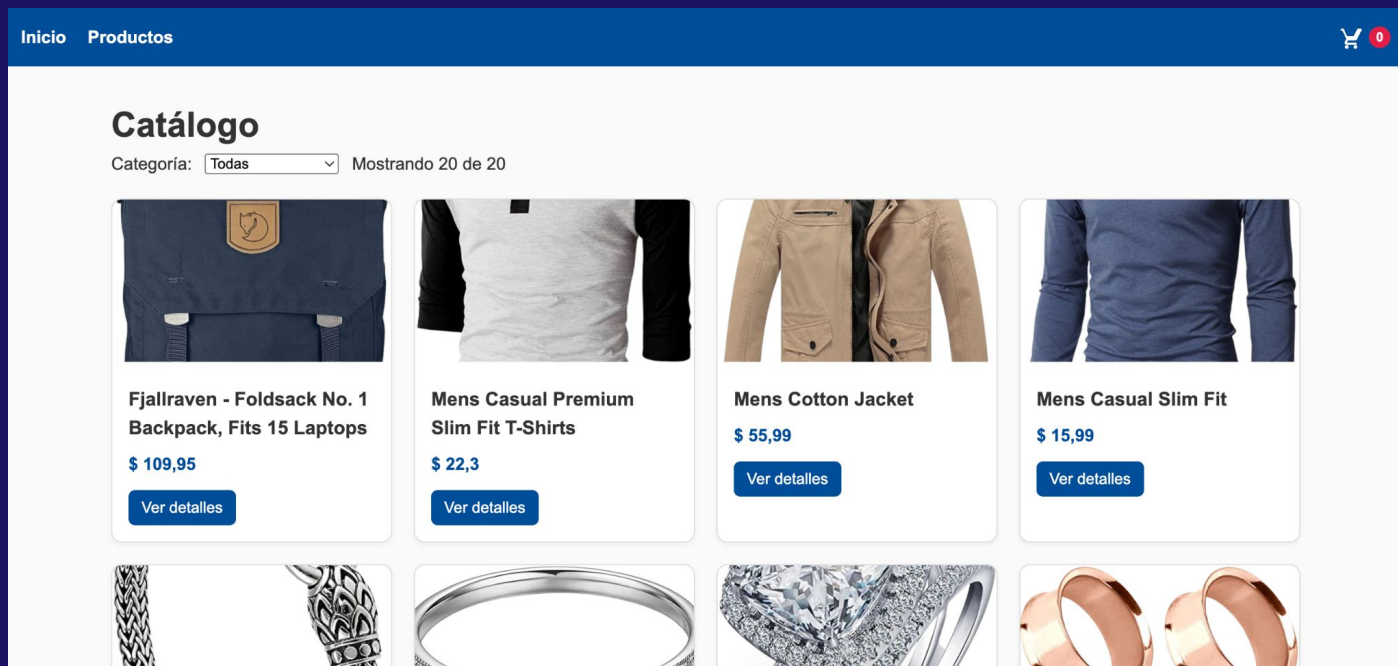
En ocasiones, querrás anidar tus propios componentes

```
<Card>  
  <Avatar />  
</Card>
```

Al anidar contenido dentro de una etiqueta JSX, el componente padre recibirá ese contenido a través de una prop llamada `children`. En el ejemplo a continuación, el componente `Card` recibe una prop `children` con el valor de `<Avatar />` y lo renderiza dentro de un `div` contenedor: [ejemplo](#)

Pensar en componentes

Pensemos en el catálogo de un ecommerce. ¿Qué componentes podríamos definir?



Pensar en componentes

Pensemos en el catálogo de un ecommerce. Qué componentes podríamos definir?

-Podríamos pensar en componentes que sean reutilizables en diferentes partes del sitio, tanto genéricos de UI como propios de la tienda.

Ejemplos de genéricos

- Navbar
- Botones
- Spinner/Loader de carga
- Alertas
- Footer

Pensar en componentes

Ejemplos propios de un ecommerce

- ProductCard** → muestra producto individual
- ProductList** → lista de productos
- CategoryFilter** → filtro por categoría
- SearchBar** → búsqueda
- PriceFilter** → filtro por precio
- CartItem** → producto dentro del carrito
- CartSummary** → subtotal, total, envío
- QuantitySelector** → sumar/restar cantidad
- OrderSummary** → resumen de compra
- CheckoutForm** → datos del comprador

Pensar en componentes

Veamos un Navbar de ejemplo

Consideremos pasarle por props el título, el nombre de cada sección y la cantidad del carrito

```
import './Navbar.css'

function Navbar({ titulo, links, cantidadCarrito }) {
  return (
    <nav className="navbar">
      <h2>{titulo}</h2>

      <ul className="nav-links">
        {links.map((link, index) => (
          <li key={index}>{link}</li>
        ))}
      </ul>

      <div className="cart">🛒 {cantidadCarrito}</div>
    </nav>
  )
}

export default Navbar
```


Pensar en componentes

Veamos un Navbar de ejemplo

Llamamos el componente Navbar con sus props en la App

```
import './App.css'
import Navbar from './components/Navigation/Navbar'
function App() {

  return (
    <>
      <Navbar titulo="Mi App" links={['Inicio', 'Productos', 'Contacto']} cantidadCarrito={0}>
    </>
  )
}

export default App
```

Pensar en componentes

Veamos un Navbar de ejemplo

Le podemos añadir estilos de css con la prop className e importando un archivo css como ya conocemos

```
import './Navbar.css'

function Navbar({ titulo, links, cantidadCarrito }) {
  return (
    <nav className="navbar">
      <h2>{titulo}</h2>

      <ul className="nav-links">
        {links.map((link, index) => (
          <li key={index}>{link}</li>
        ))}
      </ul>

      <div className="cart">🛒 {cantidadCarrito}</div>
    </nav>
  )
}

export default Navbar
```

Pensar en componentes

Ejemplo de ProductCard

Para este ejemplo recibe las props: nombre, precio, imagen, descripción y stock

```
import './Card.css'

function ProductCard({ nombre, precio, imagen, descripcion, stock }) {
  return (
    <div className="card">
      <img src={imagen} alt={nombre} className="card-img" />

      <h3>{nombre}</h3>
      <p>{descripcion}</p>
      <p>${precio}</p>
      <p>Stock: {stock}</p>

      <button>Agregar al carrito</button>
    </div>
  )
}

export default ProductCard
```

Pensar en componentes

Llamada en App

```
function App() {  
  
  return (  
    <>  
      <Navbar titulo="Mi App" links={['Inicio', 'Productos', 'Contacto']} cantidadCarrito={0}  
      <div className="catalogo">  
        <ProductCard  
          nombre="Notebook"  
          precio={350000}  
          imagen="https://via.placeholder.com/200"  
          descripcion="16GB RAM"  
          stock={5}  
        />  
  
        <ProductCard  
          nombre="Mouse"  
          precio={15000}  
          imagen="https://via.placeholder.com/200"  
          descripcion="Mouse inalámbrico"  
          stock={10}  
        />  
      </div>  
    </>  
  )  
}  
  
export default App
```

Pensar en componentes

Veamos un Button de ejemplo

Consideremos pasarle por props texto, size, color, efecto

```
import './Button.css'

function Button({ texto, color, size, efecto }) {
  return (
    <button className={` ${color} ${size} ${efecto}`}>
      {texto}
    </button>
  )
}

export default Button
```

Luego podríamos llamarlo en otro componente, ej en ProductCard

Actividad 1

Siguiendo los ejemplos vistos en clase, crear el proyecto base de react y:

- Crear componente Navbar
- Crear componente Footer
- Crear componente ProductCard

Notas:

Aplicarles estilos css y pasarles props de modo que sean reutilizables



React router dom

¿Para qué sirve?

Permite que una app React tenga varias vistas o páginas sin recargar el navegador.
👉 React sigue siendo una sola aplicación, pero cambia qué componente se muestra según la URL.

Uso de React router dom

Instalación

Ejecutar: `npm install react-router-dom`

1- Envolver la App: BrowserRouter habilita navegación.

```
import { BrowserRouter } from 'react-router-dom'

<BrowserRouter>
  <App />
</BrowserRouter>
```

2-Definir rutas: cada ruta conecta URL con componente.

Uso de React router dom

```
import { Routes, Route } from 'react-router-dom'

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/productos" element={<Productos />} />
</Routes>
```

[Ver ejemplo de código](#)

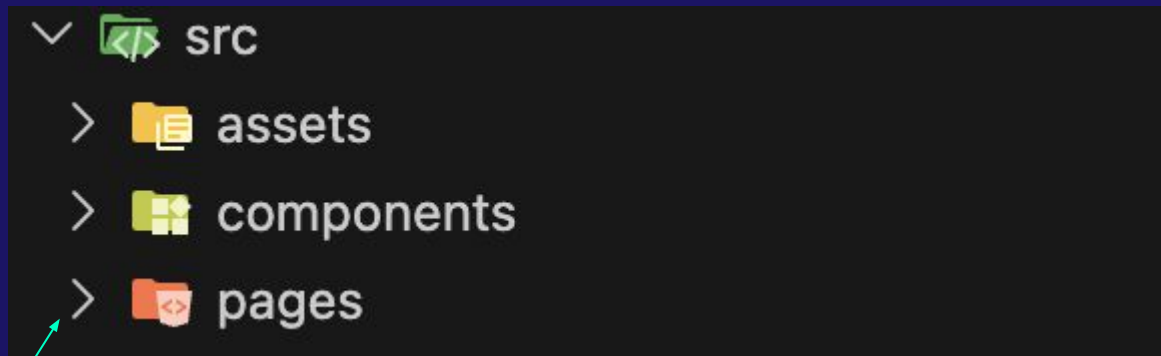
Especifico la ruta de cada elemento.
Para este caso defino la page Products, donde voy a mostrar el catálogo

```
import './App.css'
import { Routes, Route } from 'react-router-dom'
import Navbar from './components/Navigation/Navbar'
import Products from './pages/Products'

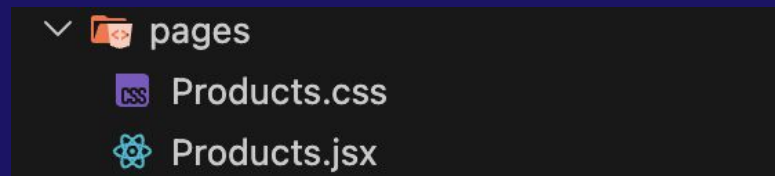
function App() {
  return (
    <>
      <Navbar
        titulo="Mi App"
        links={[
          { nombre: 'Inicio', ruta: '/' },
          { nombre: 'Productos', ruta: '/productos' },
          { nombre: 'Contacto', ruta: '/contacto' }
        ]}
        cantidadCarrito={0}
      />

      <Routes>
        <Route path="/productos" element={<Products />} />
      </Routes>
    </>
  )
}

export default App
```



Creo la carpeta pages dentro de src
Donde voy a tener las distintas
secciones del sitio



Dentro de Products.jsx voy a renderizar
cada ProductCard

Uso de Link

Permite **navegar entre páginas dentro de una app React sin recargar el navegador**

Con la etiqueta `<a> <a/>` de enlaces, se recarga toda la página y se vuelve a pedir todo al servidor

Con Link

```
<Link to="/productos">Productos</Link>
```

React cambia solo el componente visible, sin recargar.

Ejemplo con Navbar

```
import './Navbar.css'
import { Link } from 'react-router-dom'

function Navbar({ titulo, links, cantidadCarrito }) {
  return (
    <nav className="navbar">
      <h2>{titulo}</h2>

      <ul className="nav-links">
        {links.map((link, index) => (
          <li key={index}>
            <Link to={link.ruta}>{link.nombre}</Link>
          </li>
        ))}
      </ul>

      <div className="cart">🛒 {cantidadCarrito}</div>
    </nav>
  )
}

export default Navbar
```

```
<Navbar
  titulo="Mi App"
  links={[
    { nombre: 'Inicio', ruta: '/' },
    { nombre: 'Productos', ruta: '/productos' },
    { nombre: 'Contacto', ruta: '/contacto' }
  ]}
  cantidadCarrito={0}
/>
```

Actividad 2

1-Crear las diferentes pages del ecommerce ej: Home, Productos, Contacto

src/pages/Home.jsx

src/pages/Productos.jsx

src/pages/Contacto.jsx

2- Utilizar React router dom y llamar estas pages desde la App.

Ej:

/ → Home

/productos → Productos

/contacto → Contacto

3- Dentro del componente Productos renderizar las cards de productos creadas en la actividad anterior.

4- Actualizar el componente Navbar para utilizar Link de react router dom.



Consumo de APIs

¿Qué es consumir una API?

Es pedir datos a otro sistema desde nuestro frontend.

Axios: ¿Qué es?

Es una librería de JavaScript que se usa para hacer peticiones HTTP a una API desde el frontend.

Permite que React se comuniquen con el backend.

Sirve para:

- Obtener datos
- Enviar datos
- Actualizar datos
- Eliminar datos

Axios: ¿Qué es?

Ejemplo: pedir productos

```
axios.get('http://localhost:3000/productos')
```

¿Por qué se usa mucho?

Porque simplifica el trabajo frente a fetch. → hay que convertir la respuesta manualmente con `res.json()`

Ventajas

- sintaxis más simple
- respuesta directa en `response.data`
- manejo de errores más claro

Axios: Instalación

Se ejecuta `npm install axios`

¿Cómo se usa?

Cada vez que necesitemos interactuar con el backend utilizamos los verbos de HTTP y llamamos a los endpoints definidos.

Ejemplo:

`axios.get('http://localhost:3000/productos')`

`http://localhost:3000` → servidor backend

`/productos` → endpoint definido en backend

Ese endpoint corresponde a algo como: **`router.get('/productos', obtenerProductos)`**